

計算機程式設計 (二)

期末自主學習專案作業

學號：S11459023

姓名：張榮豐

班級：資工一

主題：STL 與基礎資料結構入門

GitHub Repo: <https://github.com/JAY-580/Project-Week-18.git>

《目錄》

《目錄》

一、自主學習主題與目標

二、Template 與 STL 的關係

三、常見STL容器用途

四、第二周小專題設計說明

五、主要程式設計架構與重要程式片段

七、GitHub Repository 與結構檔案說明

八、遇到的問題與解決方法

九、學習心得與未來可延伸方向

【生成式人工智慧使用說明】

一、自主學習主題與目標

本次自主學習主題為「STL 與基礎資料結構入門」。課程中介紹了 C++ Template 的概念，而 STL (Standard Template Library) 正是 Template 技術的重要應用。因此本次作業希望透過自主學習，進一步了解 STL 容器以及相關演算法的使用方式。

本次學習的主要目標如下：

1. 理解 Template 與 STL 之間的關係。
2. 熟悉常見 STL 容器的用途與操作方式。
3. 學習使用 STL 演算法處理資料。
4. 練習選擇適合的資料結構解決問題。
5. 建立未來學習資料結構課程的基礎能力。
6. 完成一個整合 STL 與 Template 技術的實際程式專題。

二、Template 與 STL 的關係

Template 是 C++ 中非常重要的泛型程式設計技術。它允許函式或類別在不指定資料型別的情況下進行設計，使同一份程式碼可以應用於不同型別的資料。

它可以同時處理：

- int
- float
- double
- string

等不同型別的資料，而不需要分別撰寫多個版本。

STL 的全名為 Standard Template Library，中文稱為標準模板函式庫。STL 中的大部分容器與演算法都建立在 Template 的基礎上。例如：

```
vector<int>    vector<double>    vector<string>
```

都是同一個 vector 類別透過 Template 所產生的不同型別實例。

因此可以說：

Template 是 STL 的基礎技術，而 STL 則是 Template 的實際應用。

透過 STL，程式設計師可以快速使用已經實作好的資料結構與演算法，不需要從零開始撰寫，提高開發效率與程式品質。

三、常見STL容器用途

類別	範例	說明
Vector	<i>vector<int> numbers;</i>	● 可動態調整大小 ● 支援隨機存取 ● 適合儲存大量資料
Stack	<i>stack<int> s;</i>	● 括號配對檢查 ● 函式呼叫管理 ● Undo 功能
Queue	<i>queue<string> q;</i>	● 排隊系統 ● 工作排程 ● 模擬服務櫃台
Map	<i>map<string, int> score;</i>	● 快速查詢 ● Key 不可重複 ● 自動排序

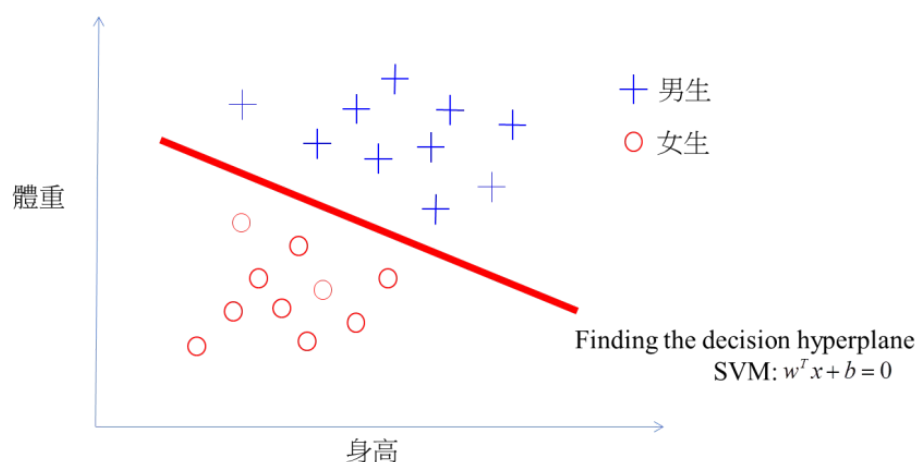
支持向量機 SVM 是一種監督式機器學習演算法，主要用於分類問題。核心概念是在不同類別的資料之間尋找一條最佳分界線

(Hyperplane)，並使分界線與兩側資料之間的距離 (Margin) 最大化，以提高模型對未知資料的判斷能力。

我參考了網上一篇文章，作者Tommy Huang推導並詳細解釋了SVM，我決定引用他的例子來解釋SVM。

他指出“「如何只用身高體重就來判斷是男生還是女生」。分類男生和女生兩類，特徵資料只有「身高」和「體重」。這邊我先隨便舉男生有10組資料，女生也是10組資料。不一定是直線，有可能是曲線，這邊只會提到直線部分，不同的演算法都是在不同的假設或是條件下去找**那條分類的線**。比如說高斯分類器就是在利用兩組資料的高斯機率分布/高斯概似函數(Gaussian likelihood function)，去判斷誰的後驗機率(posteriori probability)/概似函數值較大，就判給哪一類別。SVM則是去假設有一個hyperplane($wTx+b=0$)可以完美分割兩組資料，所以SVM就是在找參數(w和b) 讓兩組之間的距離最大化。”

[3]



在數學上，SVM 的本質是一個最佳化問題。需要在滿足分類條件下找出最大間隔，因此我們可利用微積分（二）所學的 Lagrange Multipliers 處理限制條件，進而求得最佳分類邊界。這也是 SVM 與我們課程內容的重要連結。

當資料無法用直線完全分開時，SVM 還可以利用 Kernel（核函數）將資料映射到更高維空間，使原本難以分類的資料變得更容易區分，進一步提升分類效果。這是我目前瞭解的。

SVM 與 Lagrange Multipliers的關聯

在微積分（二）課程中，我們學習到 Lagrange Multipliers（拉格朗日乘數法），其主要用途是在存在限制條件的情況下尋找最佳解。例如在固定周長下求最大面積，或在特定曲面上尋找極值點。

支持向量機的核心目標是尋找能夠最大化分類間隔（Margin）的超平面。此問題同樣屬於限制最佳化問題，因為模型必須同時滿足資料正確分類的條件以及最大化間隔的要求。因此在 SVM 的數學推導中，研究者利用拉格朗日函數將限制條件與目標函數結合，再透過最佳化方法求得最佳分類邊界。

簡單來說，若以身高與體重區分男生與女生為例，SVM 不只是找到一條能夠分開兩群資料的直線，而是尋找一條與兩側資料距離最大的最佳分界線。這個「最大距離」的求解過程，便與微積分課程中的最佳化概念密切相關。

四、第二周小專題設計說明

本次專題為學生成績管理系統。

主要目的是利用 STL 容器管理學生資料，並透過排序、查詢與統計功能練習資料結構的應用。

系統提供以下功能：

1. 新增學生資料
2. 顯示所有學生資料
3. 成績排序
4. 學號查詢
5. 成績統計

使用者透過選單輸入指令即可操作系統。

本專題主要使用 vector 作為資料儲存容器。

選擇 vector 的原因包括：

- 可動態擴充容量
- 容易新增資料
- 可直接搭配 `sort()` 排序
- 適合作為 STL 入門練習

五、主要程式設計架構與重要程式片段

Student 結構

```
struct Student
{
    string id;
    string name;
    int score;
};
```

Template Function

```
template <class T>
T getMax(T a, T b)
{
    return (a > b) ? a : b;
}
```

```
template <class T>
T getMin(T a, T b)
{
    return (a < b) ? a : b;
}
```

排序功能

```
sort(students.begin(), students.end(),  
    [](const Student& a, const Student& b)  
{  
    return a.score > b.score;  
});
```

此程式碼利用 Lambda Expression 實現由高分到低分排序。

統計功能

```
highest = getMax(highest, s.score);  
lowest = getMin(lowest, s.score);
```

利用 Template Function 完成最高分與最低分的計算。

此部分符合本次作業要求的 Template 延伸挑戰題。

六、執行畫面與輸出結果

以下為程式執行流程：

新增學生：

```
Enter ID: S001
Enter Name: Eric Chong
Enter Score: 95
```

列出學生：

ID	Name	Score
S001	Eric Chong	95
S002	John Smith	80

查詢學生：

```
Enter ID to search: S001
```

輸出：

```
ID: S001
Name: Eric Chong
Score: 95
```

統計結果：

```
Average Score : 87.50
Highest Score : 95
Lowest Score : 80
Pass Count : 2
Fail Count : 0
```

```
PS C:\homework> & "C:\Users\user\.vscode\extensions\ms-vscode.cpptools-1.32.2-win32-x64\debugAdapters\bin\Win32-
ft-MIEngine-In-qeldhkg.0vr" "--stdout=Microsoft-MIEngine-Out-jurvaucv.3kl" "--stderr=Microsoft-MIEngine-Error-0b
e-Pid-0e1foilig.ybv" "--dbgExe=C:\mingw64\bin\gdb.exe" "--interpreter=mi"

===== Student Grades Management System =====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Enter choice: 1
Enter ID: S001
Enter Name: Eric Chong
Enter Score: 95
Student added successfully.

===== Student Grades Management System =====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Enter choice: 1
Enter ID: S002
Enter Name: John Smith
Enter Score: 80
Student added successfully.

===== Student Grades Management System =====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Enter choice: 2
ID      Name      Score
-----
S001    Eric Chong  95
S002    John Smith  80

===== Student Grades Management System =====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Enter choice: 4
Enter ID to search: S001

Student Found
ID: S001
Name: Eric Chong
Score: 95

===== Student Grades Management System =====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Enter choice: 5

===== Statistics =====
Average Score : 87.50
Highest Score : 95
Lowest Score : 80
Pass Count : 2
Fail Count : 0

===== Student Grades Management System =====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
Enter choice: |
```

七、GitHub Repository 與結構檔案說明

GitHub Repo: <https://github.com/JAY-580/Project-Week-18.git>

我將程式拆分為 main.cpp 和 project.h 兩個檔案。

main.cpp 會包含 main function，主要工作是 display，結合並執行其他 function。而 project.h 包含了很多主要 function 的 definition 和計算。

其中 README.md 是專案說明文件，包含編譯與執行方式。

八、遇到的問題與解決方法

在開發本專題的過程中，我遇到了幾個問題。第一個問題是姓名輸入。原本使用：`cin >> name;` 但當輸入包含空白的姓名時，例如：Eric Chong 程式只會讀取 Eric。後來改用：`getline(cin, name);`

成功解決問題。第二個問題是輸入錯誤資料導致程式異常。例如輸入字串作為成績：abc 會造成 cin 進入錯誤狀態。因此加入：

```
cin.clear();  
cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

進行錯誤處理。第三個問題是學號重複。為避免資料重複，新增學生時會先檢查是否存在相同學號，若有則拒絕新增。

九、學習心得與未來可延伸方向

透過本次自主學習作業，我對 STL 的使用方式有更深入的了解。過去在撰寫程式時，大多使用傳統陣列儲存資料，而 STL 提供了更加方便且強大的容器，可以大幅降低程式撰寫難度。

此外，我也第一次實際將 Template Function 應用在完整專題中。透過 getMax() 與 getMin() 的實作，我理解了泛型程式設計的優點，也知道 STL 為何能夠支援不同型別的資料。

在未來，我希望能進一步學習：

1. Linked List
2. Tree
3. Graph
4. Hash Table
5. Dynamic Programming

等更進階的資料結構與演算法。

我認為本次專題不僅讓我熟悉 STL 的基本使用方式，也為未來學習資料結構課程建立了良好的基礎。透過實際撰寫程式與解決問題的過程，我對 C++ 程式設計有了更深刻的理解與興趣。

【生成式人工智慧使用說明】

本報告適度使用生成式人工智慧（以下簡稱AI）進行撰寫，所有由

AI 提供之內容皆經過本人查核、篩選與修正最後才納入本報告。以

下說明本報告使用 AI 的範圍：

1. 本報告部分**文字內容**於撰寫過程中參考生 AI 所提供的建議。然而最終內容全部由本人依據文獻資料進行修改與整合，包括段落組織、內容摘要以及文字表達。
2. 本人將 AI 視為學習與寫作輔助工具。**所有引用 AI 協助生成之內容均經過人工檢查與修正**，並由本人負責最終內容之正確性與完整性。